



Planificación del Proyecto - Scrum

Temporis:

Una herramienta
accesible para el
control y la
optimización de
nuestro tiempo

1. Introducción y Metodología

Para el desarrollo de **Temporis**, se adopta el marco de trabajo ágil **Scrum**, adaptando estratégicamente sus eventos, ceremonias y artefactos a un entorno de **desarrollo individual (Solo-Scrum)**. El ciclo de vida del proyecto se divide de forma estricta en **Sprints fijos de dos semanas de duración**.

Esta ventana temporal garantiza un ritmo de inspección y adaptación constante, permitiendo desplegar **incrementos de software potencialmente entregables** tras cada ciclo. La flexibilidad de este enfoque facilita la integración inmediata de las pautas de accesibilidad y las correcciones de diseño identificadas durante las fases de prueba continua, mitigando así posibles riesgos de desviación, en el alcance final.

2. Product Backlog (Pila del Producto)

El *Product Backlog* se compone de historias de usuario y requerimientos técnicos desglosados y ponderados según su valor para el **Mínimo Producto Viable (MVP)** y su impacto en la **inclusión digital**. La priorización sigue una taxonomía estricta de dependencias técnicas:

- **Prioridad Alta (Bloque de Infraestructura y Core CRUD):**
 - Aprovisionamiento del entorno de desarrollo y SDKs esenciales.
 - Capa de persistencia *cloud* y aislamiento criptográfico de datos (*Firebase*).
 - Gestión de la lógica operacional y transaccional del temporizador.
- **Prioridad Media (Lógica del Contador, Arquitectura y UI Base):**
 - Sincronización asíncrona en segundo plano (Corrutinas de *Kotlin* y Servicios nativos).
 - Estructura arquitectónica limpia siguiendo el patrón MVVM.
 - Implementación de la interfaz responsiva y soporte nativo para el Modo Oscuro.
- **Prioridad Baja (Análítica Avanzada y Optimización de la Accesibilidad):**
 - Módulo estadístico y renderizado del mapa de calor de productividad (*heatmap*).
 - Temas de interfaz adaptados a entornos de baja visión (Modo de Alto Contraste).
 - Pulido fino de microinteracciones, transiciones visuales y animaciones inclusivas.

3. Planificación de *Sprints* (*Sprint Backlog*)

Sprint 0: Inicialización de la Infraestructura Base y Modelo de Datos Cloud

- **Duración:** “2 semanas.” [desarrollado aprovechando momentos libres durante un periodo de unos dos meses y medio. Resulta complicado definir una duración concreta para esta primera fase del desarrollo debido a que, en ese momento, el tiempo libre disponible era bastante limitado, compaginando el trabajo, el desarrollo de las tareas (*relacionadas con las otras asignaturas del CFGS en DAM*) y el desarrollo del proyecto en cuestión (en la asignatura PMDM)]
- **Tareas Programadas y Desglose Granular:**
 1. Configuración inicial del proyecto mediante el uso del *IDE* Android Studio, de *Gradle* (como herramienta de Gestión de dependencias) y también el control de versiones *Git*, a través de *Github*.
 2. Vinculación y autenticación del proyecto con la consola de *Firebase*.
 3. Implementación inicial de las *Security Rules* de *Cloud Firestore*, para el aislamiento de los datos, filtrando por el *UID* de cada usuario.
 4. Diseño y codificación de las clases de datos base (*POJOs* (*Plain Old Java Object*)/*Data Classes*) en *Kotlin*: *User.java*, *Timer.java*, *TimeRegister.java* y *Post.kt*.
 5. Inicialización e integración inicial de/con *Firebase Auth* y *Firebase Firestore*.
 6. Maquetación/Programación inicial de las vistas/pantallas de: *Login*, *Register*, *Timers*, *Timer* y “*User Dashboard*”.
- **Estado de Entrega (Tareas Realizadas):** “100%” completado y verificado en entorno local, aunque con ciertos errores, sobretodo relacionados con la actualización de los datos del usuario con la sesión iniciada, tratando de sincronizar e insertar/actualizar los nuevos datos, en *Firebase Firestore*.
- **Comentario acerca del “*sprint* inicial”:** Esta primera fase del desarrollo del proyecto se realizó durante el periodo de tiempo aproximado que comprendió desde mediados de Febrero hasta mediados de Mayo (ambos del año 2025), durante el desarrollo de la propuesta inicial de aplicación para Android. Se entregó en tiempo y forma adecuados, aunque manifestando y “manteniendo” ciertos errores/disfunciones, de forma continuada.

Sprint 1: Infraestructura Base y Modelo de Datos Cloud

- **Duración:** 2 semanas.

- **Tareas Programadas:**

1. Ajuste, reparación y mejora de la configuración del proyecto.

(Deuda técnica) Durante esta primera etapa, se han solucionado los errores que seguían presentes al finalizar la *release*/versión “Temporis Pre-release v1.0.0”, consistiendo principalmente en la funcionalidad para actualizar los datos de *User* en *F. Firestore*.

(Deuda técnica) Ajuste de la apariencia estética. Corregimos el límite de *scroll* vertical (en la lista de contadores). Y mejoramos/rediseñamos la apariencia de las tarjetas (o *card*) utilizadas para presentar los diferentes temporizadores/contadores.

2. (Mejora) Resestructuración, preparación y mejora de la interfaz de la aplicación, para poder añadir las nuevas funcionalidades y secciones.
3. (Mejora) Inicialización de las nuevas vistas/secciones y modificación/ajuste de las preexistentes.
4. (Deuda técnica) Ajuste de las *Security Rules* de *Cloud Firestore*, para el aislamiento de los datos, filtrando por el UID de cada usuario. El error a la hora de actualizar los datos del usuario se debía a un desajuste en la definición de las reglas de seguridad, relacionadas con la clase usuario.
5. (Mejora) Ajuste del diseño y codificación de las clases de datos base (*POJOs (Plain Old Java Object)/Data Classes*), codificadas en Java o *Kotlin*: *User.java*, *Timer.java*, *TimeRegister.java* y *Post.kt*. En nuestro caso, la única clase que se encuentra programada en *Kotlin* es *Post.kt*.
6. (Mejora) Ajuste de la integración y de la funcionalidad de *Firebase Auth* y *Firebase Firestore*.
7. (Mejora) Rediseño, reestructuración y mejora de las vistas existentes: *Login*, *Register*, *Timers*, *Timer* y “*User Dashboard*”.
8. (Mejora) Inicialización preliminar de las nuevas vistas (vacías) y ajuste de las redirecciones necesarias para poder acceder a estas.
9. (Deuda técnica) Reorganización de la estructura de paquetes del proyecto según los principios de *Clean Architecture* (directorios: *adapter*, *model*, *repository*, *ui* y *util*).
10. (Deuda técnica) Implementación del patrón *Repository* (*TimerRepository.kt*, *UserRepository.kt* y *PostRepository.kt*) para la abstracción de operaciones *CRUD* con *Firestore*.
11. (Deuda técnica) Desarrollo de los *ViewModels* (ej: *TimerViewModel.kt*) con control de estado mediante *MutableStateFlow*.
12. (Mejora) Despliegue y mejora del motor de cuenta atrás del temporizador mediante hilos asíncronos eficientes (*Kotlin Coroutines*).

- **Estado de Entrega (Tareas Realizadas):** Se han completado y verificado en base a nuestra definición de “hecho”.
- **Comentario adicional:** En una mayoría de las tareas definidas en el tablero *Kanban*, también se ha definido una historia de usuario, redactada con el lenguaje estructurado *Gherkin*, en el interior de cada tarjeta (o *card*).

Sprint 2: Diseño de Interfaces Responsivas y Accesibilidad Universal (LSE/WCAG)

- **Duración:** 2 semanas.
- **Tareas Programadas:**
 1. Diseño y maquetación del árbol de vistas XML utilizando componentes adaptativos (*ConstraintLayout*).
 2. Implementación y unificación del sistema de diseño visual (estilos de botones redondeados, gradientes y jerarquía tipográfica).
 3. Creación e integración de los recursos de color para soporte nativo de **Modo Oscuro** y **Modo de Alto Contraste**.
 4. Inyección de atributos **contentDescription** en todos los componentes interactivos e imágenes para la correcta verbalización en el lector de pantallas *TalkBack*.
 5. Ajuste del escalado tipográfico adaptativo dinámico en unidades *sp* (hasta 30sp) previniendo el desbordamiento o colapso de las vistas de la interfaz.
- **Estado de Entrega (Tareas Realizadas):** Completado. Las vistas de configuración de accesibilidad y selección de modos son completamente funcionales. Código integrado en la rama principal (*main*) tras haber superado primero las pruebas manuales y posteriormente, también los *test* unitarios y de integración.
- **Comentario adicional:** En una mayoría de las tareas definidas en el tablero *Kanban*, también se ha definido una historia de usuario, redactada con el lenguaje estructurado *Gherkin*, en el interior de cada tarjeta (o *card*).

Sprint 3: Documentación, Control de Calidad (QA), Inspección Estática y Cierre

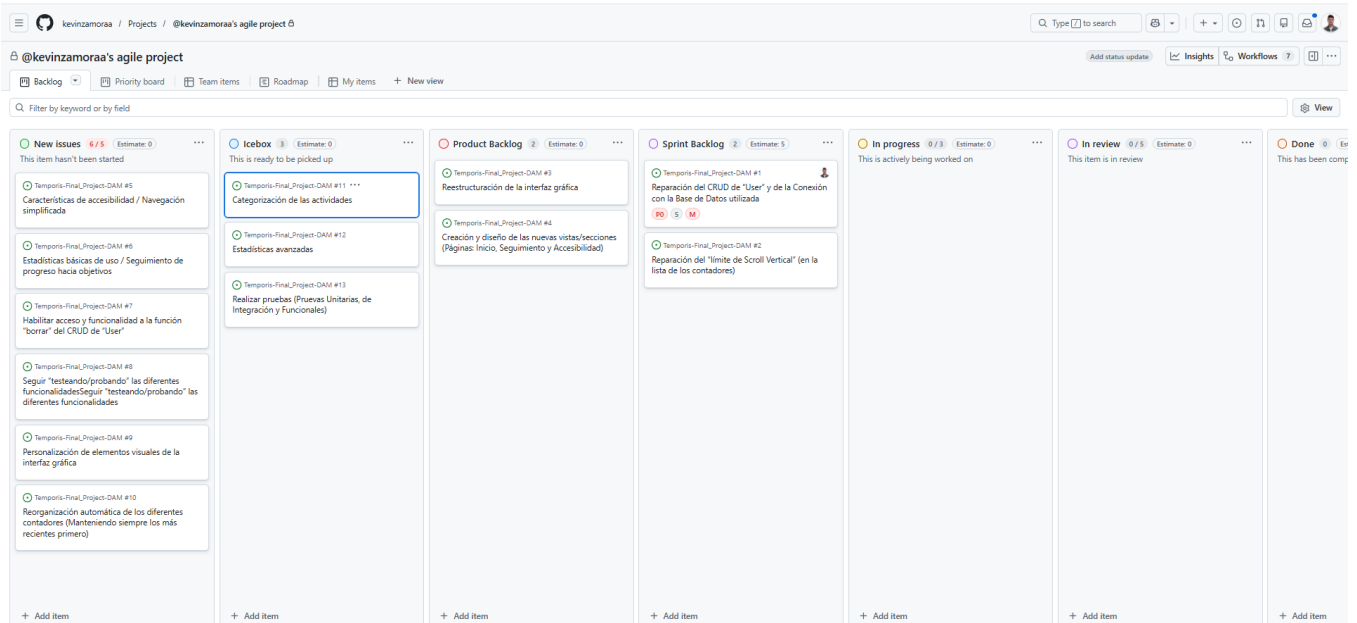
- **Duración:** 2 semanas.
- **Tareas Programadas:**
 1. Configuración del servidor remoto mediante Github Action, de la exportación de los datos hacia **SonarQube** y de la ejecución del plugin de Gradle, para análisis estático de código.
 2. Refactorización y limpieza de la complejidad ciclomática en ViewModels y erradicación de *code smells* detectados.
 3. Escritura y ejecución de baterías de pruebas unitarias con JUnit e instrumentales de interfaz con el framework Espresso.
 4. Escritura y ejecución de baterías de pruebas de integración entre componentes.
 5. Comprobación manual de la accesibilidad móvil desde un *smartphone* físico.
 6. Compilación del artefacto final corregido y redacción de las memorias técnicas y de control de calidad.
- **Estado de Entrega (Tareas “Realizadas”):** En proceso. El objetivo de la cobertura de código que queremos validar en un futuro, debería de encontrarse por encima del 85% y con la pasarela de calidad (*Quality Gate*) aprobada.
- **Comentario adicional:** En una mayoría de las tareas definidas en el tablero *Kanban*, también se ha definido una historia de usuario, redactada con el lenguaje estructurado *Gherkin*, en el interior de cada tarjeta (o *card*).

PD: Durante el desarrollo de alguno de los *sprints* definidos, hemos sufrido algunos pequeños retrasos, debido principalmente a la limitada disponibilidad de tiempo operativo, aunque hemos sido capaces de reajustar los flujos de trabajo y continuar con el desarrollo favorable del proyecto.

4. Tablero Kanban (Herramienta de Gestión)

Para el seguimiento visual, se ha utilizado un tablero **Kanban**, creado y administrado mediante **GitHub Projects**, en el cual, las tareas, han ido transitando/desplazándose por/a través de las columnas definidas: *New issues*, *Ice Box*, *Product Backlog*, *Sprint Backlog*, *In Progress*, *In Review* y *Done*. Esto me permite detectar cuellos de botella en la fase de diseño de interfaces.

Estado inicial:



Estado actual (a 27/5/2026) :

